Garden Group - Delivery 3

Jerrel, Ahmet, Denys and Maksym

Date: 20 October 2026

GitHub repository: NOT PROVIDED. Add the real repository URL before submission.

1. Application

This application is a service desk for Garden Group. Employees can log in, report an incident and check their own tickets. Service desk employees can manage all tickets and employee accounts. The dashboard shows the number and percentage of open tickets, resolved tickets and tickets closed without a solution.

The application uses C#, ASP.NET Core MVC and MongoDB. The interface is deliberately simple: a dashboard, a ticket list, ticket forms and employee management. The same layout, ticket list and dashboard are reused for both roles.

The supplied starting ZIP already contained the employee/ticket models, repository interfaces, basic CRUD methods, MongoDB setup and status-count aggregation methods. It did not contain a working login or the application screens. The completed version extends that work instead of treating the original ZIP as a finished application.

2. Contribution record

The ZIP contained no reliable author names or Git history. The following is a suggested division for the four members to review and demonstrate. It is not a verified statement of who developed the existing files. Replace it with the actual work distribution before submission, using commits and the group's own records. AI assistance was used to extend, document and test this version; members should follow their course's disclosure rules and understand any work they submit.

Member	Suggested components to review and demonstrate	Separate additional feature	Actual development contribution
Jerrel	Login, role checks, employee dashboard and ownership rules	AND/OR ticket search (`TicketSearchService`)	To be confirmed by the group
Ahmet	Employee repository, employee forms, account validation and indexes	Priority sorting (`PrioritySortService`)	To be confirmed by the group
Denys	Ticket CRUD screens, ticket repository, status/date workflow	Ticket transfer (`TicketTransferService`)	To be confirmed by the group
Maksym	Shared layout, demo data, setup and verification	Filtering loaded tickets (`TicketFilter` class in JavaScript)	To be confirmed by the group

Each member should record what they actually designed, implemented, changed and tested. Suggested ownership alone does not prove an individual contribution. The original classmate's contribution should be credited once the group confirms its author.

3. Implementing the Delivery 1 model

The database design was already submitted in GardenGroupDelivery1.pdf. This application follows that MongoDB document model rather than introducing a new model. Employee and Ticket remain separate collections. Name, Contact and Location are embedded inside Employee; Dates is embedded inside Ticket. ReportedByEmployeeId and the nullable AssignedToEmployeeId reference Employee documents using

ObjectIds. SolvedAt and ClosedAt remain separate, as explained in Delivery 1.

Delivery 1 already explains the main alternatives: one Employee entity with a role instead of separate employee types, references instead of copying employee details into every ticket, and separate tickets instead of a growing employee ticket array. Those choices are retained here. The completed application adds the screens, permissions and queries needed to use that design.

There are small naming details in the implementation. A camelCase convention maps C# properties to database fields, Id maps to _id, and the password hash field is named passwordHash (labelled PWHash in the Delivery 1 diagram). The actual exports use name.firstName/name.lastName, title and statusInfo; explicit field mappings connect these to the C# names. The event date uses statusInfo.dateReported. DATABASE.md records the exact stored names; it is an implementation reference, not a replacement ERD. The original exports lacked priority, type and deadline. Suggested type and priority values have now been added to the distributed JSON and local copy using the incident title; deadlines remain unset. Classification is editable and does not claim verified business impact. The demonstration imports the real exported data into a separate local database and does not change Atlas. The exported bcrypt-style hashes were incomplete, so only the approved local copy gets new demo password hashes.

The supplied Garden.Employee.json and Garden.Ticket.json contain 120 employees and 150 tickets. Their IDs, names and ticket relationships are retained in the local import. Reporter/assignee existence is checked by the application because MongoDB does not enforce ObjectId references as SQL foreign keys. Deleting an employee referenced by a ticket is blocked. These are implementation checks around the existing relationships.

4. Implementation choices

MVC and repositories

The starting project mixed MVC startup routing with default Razor Pages. The completed application uses MVC consistently: a controller handles the request, a repository queries MongoDB, and a Razor view displays the result. Repositories keep database code out of the views. Small services hold the separate additional features and ticket status rules.

The MongoClient is registered once and reused. Repositories use asynchronous database calls so a request does not block a thread while waiting for MongoDB. Form view models describe the fields a user may submit instead of binding a complete employee document, which includes a password hash, directly from a request.

Login and permissions

Passwords are stored using ASP.NET's PasswordHasher rather than plaintext or a homemade hash. Login checks the submitted password against the stored hash. A successful login creates an encrypted authentication cookie with the employee ID and role. Invalid credentials receive one generic error message.

Permissions are checked on the server as well as reflected in the navigation. A regular employee's reporter ID is taken from their signed-in account, not trusted from the form. The ticket queries and details page limit them to their own reported tickets. Service desk controllers/actions are protected by role checks. A valid anti-forgery token is required for changes.

Sessions are checked against the employee document on each authenticated request. A deleted account, changed role or changed password invalidates the old session. This adds a small database read per request but keeps changed access from remaining active until the cookie expires.

Dashboard aggregation

The database performs the counting through an aggregation pipeline. For a regular employee, a match stage restricts the tickets to that reporter. A group stage counts the three outcome categories. Service desk users

use the same logic without the reporter restriction. One shared view model calculates percentages, and the same Razor view displays them.

Returning grouped counts avoids transferring every ticket just to count dashboard statuses. The total is the sum of the three categories. If there are no tickets, percentages are zero rather than causing division by zero. With the supplied data, the service desk sees 150 tickets: 74 open (49.3%), 49 resolved (32.7%) and 27 closed without resolution (18%).

Status and deletion

SolvedAt and ClosedAt are separate. A closed ticket may be a duplicate or no longer needed without the problem being solved. Resolving a ticket records SolvedAt. Closing it records ClosedAt and keeps any existing solved date. Reopening clears both dates because this version does not keep a full event history.

The supplied user stories say to close tickets instead of deleting them, while the rubric asks for ticket CRUD including delete. Closing is therefore the ordinary action. A separate service desk-only permanent delete screen, with explicit confirmation, also supports the rubric's CRUD requirement. This interpretation should be checked with the teacher.

Employee deletion is restricted if tickets reference the account, and a service desk user cannot delete their own account or remove their own service desk role. Duplicate emails are rejected by validation and by a unique database index.

5. Additional features

Search

TicketSearchService searches both subject and description. For every word it creates a subject-or-description condition. These word conditions are combined with AND or OR according to the selected option. Search is case-insensitive and input is escaped so characters such as a dot or star are treated literally. Results default to newest first. Ownership is combined with the complete search filter, so OR search cannot expose another employee's tickets.

Priority sorting

PrioritySortService uses the order Critical, High, Normal, Low. Alphabetical sorting would be wrong. Users can choose high-to-low or low-to-high. Ties use the reported date and document ID. Sorting happens before the visible page is selected, so it applies to all matching tickets rather than just the current page.

Transfer

TicketTransferService provides a separate transfer action. It checks that the target is a different, existing service desk employee. It changes the assignee while keeping the incident details, status and dates. Regular employees cannot invoke this action.

Filtering a loaded list

The TicketFilter JavaScript class filters the rows already shown on the current page. Typing keywords hides rows that do not contain those words. It does not contact MongoDB or load another page. This is intentionally separate from search, which queries all matching database tickets. The screen labels the two controls differently and reports how many loaded rows remain visible.

6. Testing and limits

The application built with warnings treated as errors. Sixteen C# checks cover password verification, search-filter construction, literal escaping, priority order, dashboard percentages and ticket completion dates. Thirty HTTP checks against a separate local MongoDB database passed for login/logout, role restrictions,

dashboard scope, ticket/employee CRUD, search, sorting, transfer, anti-forgery protection and reference checks. Verification records were removed after the checks.

The interface was also checked in a browser. VERIFICATION.md contains the repeatable commands and demonstration steps. The original integration checks used a separate synthetic database. The revised version was additionally checked with the supplied exports. A read-only Atlas connection confirmed 120 employees and 150 tickets; application changes use the local imported copy.

There are some deliberate limits. Priority sorting and page selection currently happen in application memory, which is sufficient for the classroom dataset. Literal regex search is simple but is not backed by a text-search index. A larger system would need query measurements, database-side pagination and a suitable search index. The system has no full ticket audit history or simultaneous-edit conflict detection. Checks across employee and ticket documents are not a replacement for database transactions.

The separate UI prototype was not supplied, so exact visual/user-flow agreement with that prototype could not be verified. The screens implement the supplied user-story flow. The group should compare the app with its class prototype before submission.

7. Running and submitting

README.md explains the local demo script, manual MongoDB setup, demo accounts and configuration. The data import is explicit and refuses nonempty collections. The original JSON files are included in data/. The local-only reset option creates working demo password hashes without editing the source JSON or Atlas. Credentials from the starting ZIP were not copied into this package. Build files, local database files and private settings are ignored by Git.

The source package includes all application source, the solution and project files, local styles/scripts, demo-data code, checks and documentation. Dependencies are restored through NuGet. MongoDB and the .NET runtime/SDK must be installed separately.

Before submission, add the real GitHub URL, confirm the contribution record and individual feature owners, and make sure each member can explain their components. The package does not contain a published GitHub repository or verified authorship history.

References

- Supplied GardenGroupDelivery1.pdf: original entity/document design and explanations.

- Supplied NoSQL user stories workbooks: employee/service desk functionality and additional feature requirement.

- Supplied Rubrics Project NoSQL 2526.pdf: Delivery 2/3 acceptance criteria.

- Supplied Allowed additional individual functionalities for Project NoSQL (3).pdf: separate search, sorting, transfer and filtering choices.

- Microsoft, Cookie authentication: https://learn.microsoft.com/aspnet/core/security/authentication/cookie

- MongoDB, .NET/C# aggregation: https://www.mongodb.com/docs/drivers/csharp/current/aggregation/